

6 "gotchas" in machine learning—and how to avoid them

ML is hard and you can't plan for everything. Here are a few things I've learned and a few tips to avoid common missteps

[Brett Young](#)



Share



Comment



1 star



Created on June 5 | Last edited on June 7

Machine learning is full of potential pitfalls that can trip up even the most experienced practitioners. Whether it's deployment issues, data quality concerns, imbalanced datasets, or hardware constraints, there are numerous "gotchas" that, despite their subtlety, can be surprisingly costly.

Understanding and navigating these challenges is key to building reliable and effective ML systems. This article will explore some common pitfalls and provide practical advice on how to avoid them, as well as a few of my favorite tips and tricks for machine learning.

Here's what we'll be covering:

Table of contents

[Table of contents](#)

[Gotcha #1: Not considering deployment constraints](#)

[Gotcha #2: Silent errors](#)

[Gotcha #3: Using improper evaluation metrics for imbalanced datasets](#)

[Gotcha #4: CPU/RAM bottlenecks](#)

[Gotcha #5: Programmatic dataset splitting](#)

[Gotcha #6: Improper model usage during inference](#)



By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts. 

[Cookies Settings](#)

Accept All

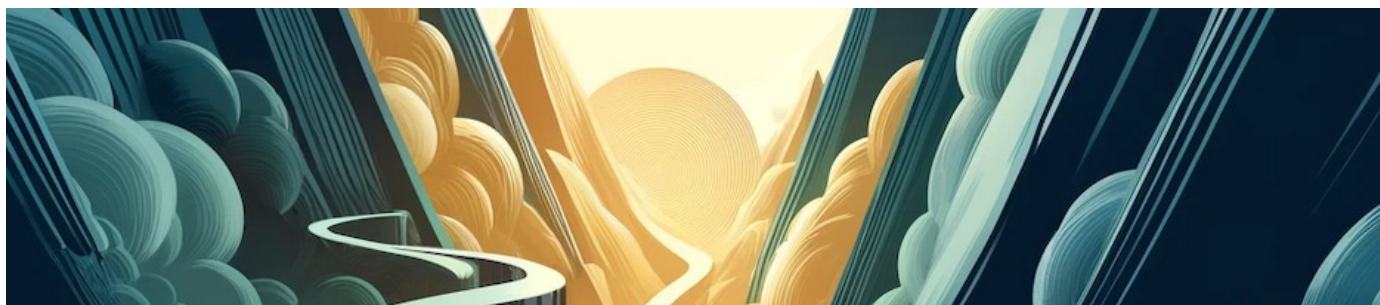
Reject All

[Leverage VSCode Python debugging](#)

[Less is more](#)

[Script logging](#)

[Conclusion](#)



Gotcha #1: Not considering deployment constraints

When building an ML system, it's helpful to start backwards from deployment. Many times, there will be constraints around final model performance, such as latency and platform compatibility.

Let's say you are building a mobile app for iOS. One major constraint is simply support for what's used in your deep learning framework of choice. For example, maybe you want to use a 3D convolutional layer with TensorFlow Lite. Does TF Lite support 3D convolution along with acceleration of Apple's metal and GPU's? If not, you will need to implement these layers from scratch, which can be a huge undertaking.

In these situations, you may be better off finding a new model architecture with existing TensorFlow Lite support. This "gotcha" can be frustrating, but by starting with deployment, you will be able to make the correct architectural adjustments early on, instead of going through the model development process and then having to redo work.

By **starting with deployment considerations**, you can make informed decisions about the choice of algorithms, model architectures, and frameworks early in the development process. This proactive approach minimizes the risk of encountering insurmountable barriers later on, ensuring a smoother transition from model development to production deployment.

Gotcha #2: Silent errors

One of the significant challenges in machine learning is the occurrence of silent errors. A

silent error occurs when model performance is negatively affected without any immediate or obvious signs. These errors can be particularly insidious because they can go unnoticed until they cause significant problems.

For example, imagine you're working with a dataset containing mislabeled images. If the model learns incorrect patterns from these mislabeled images, it will likely perform poorly when faced with real-world data, even though it might show promising results during training and validation.

To mitigate the risk of silent errors, it's crucial to **implement rigorous data validation processes**. This includes thorough initial data cleaning and ongoing data monitoring. For instance, unit tests can be applied to datasets to validate assumptions about the data. A test as simple as displaying the values returned by the dataset can save huge amounts of headache, and is generally smart practice before training a model.

By emphasizing data quality and actively seeking to detect and correct silent errors, you can significantly improve the robustness and reliability of your machine learning models, ensuring they perform well in real-world applications.

Gotcha #3: Using improper evaluation metrics for imbalanced datasets

Imbalanced data can significantly distort performance metrics, making them misleading. For instance, in a dataset where 95% of the data belongs to one class, a model that always predicts the majority class will have high accuracy but will be useless in practice. This issue is common in fields such as fraud detection, medical diagnosis, and anomaly detection, where the events of interest (e.g., fraudulent transactions, rare diseases, anomalies) are rare compared to normal cases.

To address the challenge of imbalanced data, it's essential to **use appropriate performance metrics** that provide a more accurate picture of the model's effectiveness. Metrics like precision, recall, and F1-score are more informative than accuracy in these scenarios. Precision measures the proportion of true positive predictions among all positive predictions, while recall measures the proportion of true positive predictions among all actual positives. The F1-score, which is the harmonic mean of precision and recall, provides a single metric that balances the trade-off between precision and recall.

In addition to using better metrics, various strategies can be employed to handle imbalanced data. One common approach is resampling the dataset, either by oversampling the minority class or undersampling the majority class. Oversampling techniques—such as SMOTE (Synthetic Minority Over-sampling Technique)—generate

synthetic samples for the minority class by interpolating between existing samples. This can help balance the class distribution without simply duplicating existing samples, which could lead to overfitting.

Gotcha #4: CPU/RAM bottlenecks

There's lots of focus on GPUs and VRAM, but there is usually less emphasis on CPU RAM, which is equally important during various stages of the ML pipeline, including data processing, training, and inference. Neglecting CPU and RAM can lead to performance bottlenecks and system instability, especially with large datasets or complex models.

Data processing and preparation

Data processing and preparation tasks like loading, transforming, and cleaning data are memory-intensive. Insufficient CPU RAM can slow down these operations, extending development cycles (this is another silent error). For instance, reading a large dataset into memory for preprocessing can exhaust RAM, causing the system to swap to disk and degrade performance.

Additionally, CPU speed plays a crucial role in these tasks. A faster CPU can handle data transformations and computations more efficiently, reducing the overall time required for processing. When dealing with large datasets or complex transformations, a slower CPU can become a bottleneck, further extending development time and slowing down the entire workflow.

CPU offloading

Using CPU offloading increases the demand for CPU RAM. Techniques like DeepSpeed's gradient checkpointing, which reduce GPU memory usage by recomputing activations during backpropagation, can increase the CPU's memory requirements. Ensuring sufficient CPU RAM is essential when employing such techniques.



In a recent project, I experienced significantly slow video frame reading using `imageio` compared to `OpenCV`. By switching to `OpenCV`, I dramatically increased my training speed by 3-4x and reduced my cycle time, making it much faster to try new ideas. This optimization was equivalent to tripling my "GPUs" by simply improving the data loader.

Gotcha #5: Programmatic dataset splitting

Effective dataset management is crucial for developing reliable and reproducible machine learning models. One common pitfall is the method used to split datasets into

training, validation, and test sets.

While programmatic splits are convenient, **they can lead to inconsistent test sets when new data is added**. This inconsistency invalidates previous comparisons and can obscure genuine model improvements. For instance, if you add new data to your dataset and split it programmatically, the resulting training and test sets will differ from previous iterations. This makes it impossible to accurately compare new models with older ones, as they were evaluated on different test sets. **Although using random seeds can make splits reproducible, they do not address the issue of different test sets when new data is introduced.**

Programmatic split example

A programmatic split divides your dataset into training, validation, and test sets using code each time you run your experiment. Here is an example:

```
from sklearn.model_selection import train_test_split

# Original dataset
data = ... # your dataset here
train_data, temp_data = train_test_split(data, test_size=0.3, random_state=42)

val_data, test_data = train_test_split(temp_data, test_size=0.5, random_state=42)
# Adding new data
new_data = ... # new data here
data = data + new_data # combining original data with new data

train_data, temp_data = train_test_split(data, test_size=0.3, random_state=42)

val_data, test_data = train_test_split(temp_data, test_size=0.5, random_state=42) # test
```

This approach leads to different splits each time new data is added, making comparisons between models unreliable.

Manual splitting for consistency

To ensure consistent comparisons over time, it's better to split the dataset manually (saving them as a static file) once and maintain these splits. By manually splitting your data, you create fixed training, validation, and test sets that remain unchanged even as new data is added. Consequently, you can reliably compare the performance of new models against previous ones, as they are evaluated on the same data.

Gotcha #6: Improper model usage during inference

Understanding and correctly implementing evaluation `eval()` and training `train()` modes in Torch is crucial for developing robust models.

Eval and train modes

In frameworks like PyTorch, models have two primary modes: `train()` and `eval()`

Training mode: Enables dropout and uses batch statistics in batch normalization.

Evaluation mode: Disables dropout and uses running statistics in batch normalization.

Failing to switch to `eval()` mode during validation or testing can lead to misleading performance metrics, as training-specific behaviors will still be active. This might seem obvious, but it's an easy step to forget and can cost hours of debugging time.

Temperature scaling

Temperature scaling controls the randomness of predictions by adjusting the logits before applying softmax.

Higher temperature: More uniform (less confident) predictions, useful during training.

Lower temperature: Sharper (more confident) predictions, useful during inference.

This is particularly important for large language models, where temperature settings can lead to discrepancies in evaluations and significantly affect the perceived performance of the model.

Managing eval/train modes and applying temperature scaling might seem straightforward, but forgetting these steps can lead to significant issues and wasted time.

Model export consistency

Ensuring that exported models produce similar outputs as the development model is crucial for ensuring the model you have trained performs similarly after exporting! After exporting to your desired format, run a simple test to verify that the models produce the same results. This can be easily accomplished by creating some “dummy data” and feeding it through both models.

Example with ONNX

Here's an example of how to verify model consistency after exporting a PyTorch model to

ONNX:

```
import torch
import torchvision.models as models
import numpy as np
import onnx
import onnxruntime as ort

# Use an off-the-shelf model
model = models.resnet18(pretrained=True)
model.eval()

# Create input data and export the model to ONNX
input_tensor = torch.randn(1, 3, 224, 224)
torch.onnx.export(model, input_tensor, "resnet18.onnx")

# Load and check the ONNX model
onnx_model = onnx.load("resnet18.onnx")
onnx.checker.check_model(onnx_model)

# Create an ONNX runtime session
ort_session = ort.InferenceSession("resnet18.onnx")

# Generate input data and get outputs
input_np = input_tensor.numpy()
with torch.no_grad():
    torch_output = model(input_tensor).numpy()
ort_inputs = {ort_session.get_inputs()[0].name: input_np}
ort_output = ort_session.run(None, ort_inputs)

# Compare the outputs
print("PyTorch output:", torch_output)
print("ONNX output:", ort_output[0])
assert np.allclose(torch_output, ort_output[0], atol=1e-5), "The outputs are different!"
```

In this example, a simple PyTorch model is exported to ONNX. Both the original PyTorch model and the ONNX model are fed the same data, and their outputs are compared to ensure consistency. By verifying that the outputs are similar, you can confidently deploy the exported model, knowing it behaves as expected.

Some Subtle Tips For ML Engineering

Now that we have covered many of the gotchas, I want to cover a few of my favorite tips and tricks for machine learning engineering. You'll probably be aware of many of these tips, but when I got started in ML, they were non-obvious to me.

Start with highest return actions first

In machine learning, small changes can sometimes lead to huge improvements, while large changes may yield only marginal gains. It's important to prioritize tasks based on their ratio of impact to the effort required to implement them. Think of your tasks like a rectangle where ease of implementation is the width and potential impact is the height—you want a large rectangle. Prioritizing high-impact actions that require less effort can save significant time and resources.

Sometimes, "simple" changes can be more beneficial than 'smart' changes like architectural overhauls. For example, labeling a bit of data for a few hours can sometimes have a more significant impact compared to building out a new model architecture, which may take days. By focusing on these high-return actions first, you streamline the development process and achieve better results more efficiently.

Using W&B for experiment tracking

Tracking performance metrics is an important aspect of machine learning, and having a central location to manage these metrics is incredibly helpful. Tools like Weights and Biases provide a comprehensive suite for tracking experiments, model versions, and hyperparameters all in one place. This centralization improves organization and enhances collaboration, ensuring that team members can easily share results, reproduce experiments, and build upon each other's work.

W&B allows you to log and visualize a wide range of metrics, including training and validation losses, accuracies, and custom metrics. This visibility into your experiments helps in identifying trends, diagnosing issues, and making data-driven decisions. Moreover, W&B integrates seamlessly with popular machine learning frameworks like TensorFlow, PyTorch, and Keras, making it easy to incorporate into existing workflows.

One of the primary advantages of W&B is its robust experiment tracking capabilities. By storing all experiment data in a central location, it ensures that logs and metrics are not misplaced or fragmented across different systems. This is particularly important for long-term projects where tracking performance over extended periods is essential.

Additionally, W&B's reporting features enable you to create and share interactive reports with visualizations, annotations, and insights, making it easier to communicate findings and progress.

W&B can be implemented using just a few lines of code:


```

import wandb
import torch

# Initialize W&B project
wandb.init(project="my_ml_project", entity="byyoung3")
model = ...

# Example training loop
for epoch in range(5):
    # Simulate training
    # ....
    # Log metrics
    wandb.log({"epoch": epoch, "loss": loss})

```

Track production outputs

Monitoring the performance and reliability of deployed models is just as important as monitoring during the training stages. Achieving good test set accuracy is great, but is your model actually performing well in production? Tools like Weave by Weights and Biases are super useful for this.

Weave is a lightweight toolkit for tracking and evaluating LLM applications. To use Weave, simply add the `@weave.op()` to decorate Python functions helps log and debug inputs, outputs, and traces, providing a comprehensive view of model performance. Regularly reviewing production outputs ensures models perform as expected and allows for timely updates and improvements.

Heres an example using Weave:

```

import weave # Importing the weave module
import openai # Importing the openai module

# Initialize Weave with your project name
weave.init("my-llm-project")

# Define a Weave operation
@weave.op()
def generate_response(prompt: str) -> dict:
    # Create an OpenAI client
    response = openai.Completion.create(
        engine="text-davinci-003",
        prompt=prompt,

```

```
        max_tokens=50
    )

    # Get the response from OpenAI
    response_text = response.choices[0].text.strip()

    # Returning the response as a dictionary
    return {"response": response_text}

# Use the defined Weave operation
result = generate_response("List some fruits.")
print(result)
```

Use tmux

Tmux is a ‘terminal multiplexer’ that allows you to run and monitor multiple terminal sessions efficiently. This tool is particularly useful for long-running processes, such as training models, *as it keeps sessions active even when disconnected*. You can split terminal windows, create persistent sessions, and navigate between them seamlessly, which can be particularly helpful for long-duration training runs.

Leverage VSCode Python debugging

VSCode offers powerful debugging features that are often underutilized. Breakpoints, for example, are especially useful for analyzing tensors and data shapes in ML workflows. These features support step-by-step execution and variable inspection, making it easier to identify and fix issues in your code. Leveraging VSCode's debugging tools can significantly streamline the development and debugging of machine learning models.

Less is more

One of the best strategies in machine learning is to start with simple models and gradually increase complexity. This approach helps you fully understand the problem, fosters creativity, and avoids going too far in a suboptimal direction. Beginning with a straightforward model to establish a baseline allows you to identify limitations and incrementally introduce complexity, leading to more robust models and innovative solutions.

Additionally, it can be tempting to jump to machine learning solutions, but sometimes simpler methods suffice. Being an ML engineer can bias you towards technical solutions, but it's crucial to first question whether ML is actually required. Simple, rule-based approaches or traditional statistical methods can often solve the problem effectively. By considering non-ML solutions first, you save time and resources, avoid unnecessary

complexity, and deploy faster. This dual approach of starting simple and questioning the necessity of ML can lead to more efficient and effective problem-solving.

Script logging

Logging training scripts is crucial for reproducibility and debugging. By logging each training run, including hyperparameters, code versions, and configurations, you can trace back and identify issues or performance changes. Tools like Weights and Biases (W&B) can help capture these details automatically. For example, using `wandb.run.log_code(".")` ensures that all code files in the current directory are logged, allowing you to reproduce experiments and maintain a clear record of each model version.

Effective script logging is particularly valuable for identifying bugs in previous models. If a performance drop or unexpected behavior is detected, you can review the logged scripts and configurations to pinpoint when and where the issue was introduced. This comprehensive logging not only aids in debugging but also enhances the reliability and maintainability of your machine learning systems. By maintaining detailed records of each training run, you can ensure consistent improvements and prevent the recurrence of past issues.

Conclusion

As you navigate the intricate landscape of machine learning, remember that success often lies in the balance between innovation and practicality. Focusing on both high-impact changes and efficient workflows will streamline your process and enhance your outcomes. Ultimately, the journey in machine learning is about continuous improvement and adaptation. By integrating these insights and strategies, you can build more robust, reliable, and effective systems, making meaningful advancements in your work and the broader field. Keep experimenting, learning, and refining your approach, and enjoy the exciting and rewarding path of machine learning!

Add a comment

Tags: Articles, Intermediate, Domain Agnostic



New course
LLM Apps: Evaluation

ENROLL NOW →

Weights
& Biases

All Hands

Google

Made with Weights & Biases. [Sign up](#) or [log in](#) to create reports like this one.

Never lose track of
another ML project.
Try Weights & Biases today.

SIGN UP

TRY W&B NOW



Weights & Biases

Get weekly updates with the latest ML news.

Subscribe

PRODUCTS

Dashboard | Sweeps | Artifacts | Reports | Tables

QUICKSTART

Documentation

RESOURCES

Courses | Forum | Tutorials | Benchmarks

W&B

